

RÉPONSE À APPEL D'OFFRES

Plateforme nationale de santé

SantéConnect

Rapport d'architecture NoSQL et prototype documentaire

Membres du groupe : Tayvadi PHAISAN / Mathis SILOTIA / Claude SABINOTO / Mariama DIAKHITE

Date : 01/07/2026

Sommaire

1. Introduction
2. Analyse du besoin
3. Livrable 1 — Architecture générale
4. Livrable 2 — Répartition des données
5. Livrable 3 — Justification des choix technologiques
6. Livrable 4 — Disponibilité et sécurité
7. Livrable 5 — Préparation de l'implémentation MongoDB
8. Prototype MongoDB — Modélisation des données
9. Conception des documents
10. Justification de la modélisation (Embedding / Reference)
11. Optimisation — Index
12. Requêtes MongoDB
13. Agrégations
14. Validation finale — Conclusion

(Note : pour un sommaire avec numéros de page, utiliser la fonction « Table des matières automatique » de Word une fois la mise en page finalisée — les titres sont déjà stylés en Titre 1/2/3.)

1. Introduction

Le présent rapport constitue la réponse de notre cabinet de conseil à l'appel d'offres émis par le Ministère de la Santé pour la conception de la plateforme nationale SantéConnect. Il présente l'analyse du besoin, l'architecture NoSQL retenue, la justification argumentée des choix technologiques, les mesures de disponibilité et de sécurité prévues, ainsi que la préparation détaillée de l'implémentation documentaire sous MongoDB qui fera l'objet de la première phase de développement.

SantéConnect a pour vocation de centraliser les échanges d'informations entre plus de 350 établissements de santé, environ 60 000 professionnels et 25 millions de dossiers patients, tout en garantissant une disponibilité continue, une montée en charge progressive et un niveau de sécurité conforme aux exigences réglementaires applicables aux données de santé. Ces contraintes — volumétrie considérable, hétérogénéité des données, criticité de la disponibilité et sensibilité des informations — orientent naturellement le choix vers une architecture NoSQL polyglotte plutôt que vers une solution unique.

Hypothèses de conception retenues (conformément à la consigne demandant d'explicitier toute hypothèse) :

- Le Ministère n'imposant aucune technologie, notre cabinet a évalué plusieurs familles NoSQL (documentaire, colonnes larges, clé-valeur, graphe, recherche) et retenu, pour chaque besoin métier, la technologie la plus adaptée plutôt qu'une solution générique unique.
- L'authentification applicative (identité, mots de passe, fédération) est déléguée à un fournisseur d'identité (OAuth2/OIDC type Keycloak) ; seules les sessions et jetons d'accès, à très forte volumétrie de lecture/écriture et à durée de vie courte, relèvent du périmètre NoSQL et sont pris en charge par une base clé-valeur (Redis).
- Le prototype demandé en partie 3 porte uniquement sur le périmètre documentaire (dossiers patients, consultations, prescriptions, examens) ; les autres composants (Cassandra, Redis, Neo4j, Elasticsearch) sont décrits au niveau architecture mais ne sont pas implémentés dans ce livrable.
- Les volumétries indiquées dans le cahier des charges (25 millions de dossiers, plusieurs centaines de millions de consultations) sont considérées comme des volumes cibles à horizon de plusieurs années ; l'architecture est conçue pour absorber cette croissance par scalabilité horizontale (sharding, partitionnement) sans refonte.

2. Analyse du besoin

L'analyse du cahier des charges fait ressortir cinq familles de contraintes structurantes pour le choix de l'architecture :

2.1 Volumétrie et hétérogénéité des données

La plateforme doit gérer des données très hétérogènes en structure et en volume : dossiers patients relativement stables mais riches et imbriqués (antécédents, allergies, traitements), consultations et prescriptions à structure variable selon la spécialité médicale, et surtout des flux massifs et continus issus des équipements médicaux connectés (plusieurs mesures par seconde et par dispositif, sur des centaines de milliers d'appareils à terme). Ces deux profils de données — documents semi-structurés à volume modéré et séries temporelles à très fort débit d'écriture — ne relèvent pas des mêmes optimisations de stockage.

2.2 Disponibilité et continuité de service

Une plateforme de santé utilisée quotidiennement par 350 établissements ne peut tolérer d'interruption : la disponibilité 24h/24 et 7j/7, la tolérance aux pannes serveur et l'absence d'interruption lors de la montée en charge sont des exigences non négociables. Cela impose des bases de données nativement distribuées, répliquées et sans point unique de défaillance (SPOF).

2.3 Évolutivité du schéma et de la charge

L'intégration de nouveaux établissements, de nouveaux types d'équipements connectés et l'évolution des pratiques médicales impliquent que les structures de données doivent pouvoir évoluer sans immobiliser la plateforme. Les bases relationnelles classiques, à schéma rigide et migrations coûteuses, sont pénalisantes dans ce contexte ; les bases NoSQL à schéma flexible (documentaire) ou à modèle en colonnes (wide-column) répondent mieux à ce besoin.

2.4 Sécurité et conformité réglementaire

Les données de santé sont des données à caractère personnel sensibles. L'architecture doit garantir la confidentialité, une authentification forte, une gestion fine des autorisations, la traçabilité des accès, le chiffrement des échanges, ainsi que des mécanismes de sauvegarde et de restauration fiables.

2.5 Diversité des usages fonctionnels

Le besoin recouvre des usages très différents : accès transactionnel au dossier patient, ingestion massive de mesures IoT, analyse de parcours de soins (relations entre patients, professionnels, établissements et pathologies), et production de tableaux de bord agrégés pour les autorités sanitaires. Cette diversité d'usages justifie une approche polyglotte : choisir, pour chaque besoin, la base de données dont le modèle de données et les garanties (cohérence, latence, débit) sont les plus adaptés, plutôt que de forcer l'ensemble des usages dans une technologie unique.

3. Livrable 1 — Architecture générale

L'architecture proposée est une architecture polyglotte, organisée en trois couches : une couche d'accès (utilisateurs et passerelle API sécurisée), une couche de services applicatifs (microservices métier), et une couche de persistance composée de cinq technologies NoSQL complémentaires, reliées entre elles par un bus événementiel.



Figure 1 — Schéma d'architecture générale de SantéConnect

3.1 Utilisateurs de la plateforme

- Patients : consultation de leur dossier médical, prise de rendez-vous, via portail web/mobile.
- Professionnels de santé (médecins, infirmiers, biologistes, pharmaciens) : saisie et consultation des dossiers, prescriptions, résultats d'examens.
- Ministère / autorités sanitaires : consultation des tableaux de bord et indicateurs épidémiologiques agrégés.
- Équipements médicaux connectés : émission continue de mesures (constantes vitales, dispositifs de suivi à domicile, etc.).

3.2 Passerelle et sécurité

Toutes les requêtes entrantes (hors flux IoT bruts, orientés vers le bus événementiel) transitent par une API Gateway qui assure la terminaison TLS, l'authentification via jetons OAuth2/JWT, la limitation de débit (rate-limiting) et le routage vers les microservices concernés. Le service d'authentification s'appuie sur un fournisseur d'identité fédéré et sur Redis pour la gestion à très faible latence des sessions et jetons.

3.3 Services applicatifs

- Service Dossiers patients & consultations : expose les API de lecture/écriture du dossier médical, s'appuie sur MongoDB.
- Service Ingestion IoT : consomme les flux de capteurs déposés sur Kafka et les persiste dans Cassandra.
- Service Parcours de soins : interroge le graphe Neo4j pour les analyses de trajectoires patients.
- Service Tableaux de bord / BI : agrège les données (via un mécanisme de synchronisation/ETL depuis MongoDB, Cassandra et Neo4j) dans Elasticsearch pour restitution aux autorités sanitaires.

3.4 Bases de données retenues

- MongoDB (documentaire) — dossiers patients, consultations, prescriptions, résultats d'examens.
- Cassandra (colonnes larges / séries temporelles) — mesures issues des objets connectés.
- Redis (clé-valeur) — sessions, jetons d'authentification, caches applicatifs.
- Neo4j (graphe) — parcours de soins, relations patients / professionnels / établissements / pathologies.
- Elasticsearch (recherche & analytique) — indicateurs, recherche full-text, tableaux de bord.

3.5 Flux principaux de données

- Flux transactionnel : utilisateur → Gateway → service applicatif → MongoDB (ou Neo4j) → réponse synchrone.
- Flux IoT : équipement connecté → Kafka → service d'ingestion → Cassandra (écriture massive asynchrone).
- Flux analytique : synchronisation périodique / événementielle de MongoDB, Cassandra et Neo4j vers Elasticsearch pour alimenter les tableaux de bord.
- Flux d'authentification : Gateway → service d'authentification → Redis (lecture/écriture de sessions à chaque requête).

4. Livrable 2 — Répartition des données

Le tableau ci-dessous synthétise, pour chaque module métier identifié dans le cahier des charges, la technologie NoSQL retenue et la justification associée. Le détail argumenté (qualités retenues et comparaison avec les alternatives écartées) est développé au livrable 3.

Module métier	Technologie retenue	Justification (synthèse)
---------------	---------------------	--------------------------

Module métier	Technologie retenue	Justification (synthèse)
Gestion des dossiers patients	MongoDB	Documents riches, imbriqués et de structure variable ; schéma flexible et évolutif sans migration lourde ; lecture du dossier complet en une seule requête.
Consultations et prescriptions	MongoDB	Même moteur documentaire que les dossiers patients : cohérence de modélisation, transactions multi-documents disponibles nativement (depuis MongoDB 4.0), historique naturellement imbriqué.
Authentification des utilisateurs	Redis (+ fournisseur d'identité OAuth2/OIDC)	Clé-valeur en mémoire : latence sub-milliseconde, expiration native des clés (TTL) idéale pour sessions et jetons, très fort débit de lecture/écriture.
Surveillance des objets médicaux connectés	Cassandra (ingestion via Apache Kafka)	Modèle en colonnes larges optimisé pour l'écriture massive et distribuée ; absence de SPOF (architecture peer-to-peer) ; scalabilité linéaire par simple ajout de nœuds ; adapté aux séries temporelles à très grande échelle.
Analyse des parcours de soins	Neo4j	Modèle en graphe natif pour représenter les relations patient / professionnel / établissement / pathologie ; requêtes de parcours et de proximité (Cypher) bien plus performantes qu'en documentaire ou relationnel classique pour ce type de traversées.
Tableaux de bord et indicateurs	Elasticsearch + Kibana	Moteur d'indexation inversée optimisé pour la recherche full-text et les agrégations en quasi temps réel sur de gros volumes ; s'interface nativement avec des outils de visualisation.
Autres besoins identifiés — communication inter-services	Apache Kafka (bus événementiel)	Découplage des microservices, absorption des pics de charge (buffer), rejouabilité des flux, garantit qu'aucune mesure IoT n'est perdue même en cas d'indisponibilité momentanée d'un consommateur.
Autres besoins identifiés — sauvegarde / audit	Snapshots natifs + collection d'audit (MongoDB) / CDC Kafka	Traçabilité des opérations et restauration point-in-time, détaillées au livrable 4.

5. Livrable 3 — Justification des choix technologiques

5.1 MongoDB — dossiers patients, consultations, prescriptions, examens

Adéquation au besoin métier — Le dossier médical d'un patient est un agrégat naturellement hiérarchique (identité, antécédents, consultations, prescriptions) dont la structure varie selon la spécialité et évolue dans le temps sans que l'on puisse figer un schéma définitif à l'avance.

Principales qualités dans ce contexte :

- Modèle documentaire (BSON/JSON) qui représente fidèlement la hiérarchie du dossier médical, réduisant le besoin de jointures.
- Schéma flexible : ajout de nouveaux champs ou de nouveaux types d'examen sans migration ni interruption de service.
- Réplication native (replica sets) et sharding horizontal pour absorber la croissance des 25 millions de dossiers et des centaines de millions de consultations.
- Support de transactions ACID multi-documents depuis la version 4.0, utile pour les opérations impliquant plusieurs collections (ex. création d'une consultation + mise à jour du dossier patient).

- Écosystème mature (index composés, TTL, chiffrement au niveau champ) directement exploitable pour les besoins de sécurité et de performance.

Alternatives écartées et raisons — Une base relationnelle (PostgreSQL/MySQL) imposerait un schéma rigide et de nombreuses jointures pour reconstituer un dossier complet, avec des migrations lourdes à chaque évolution ; une base orientée colonnes larges (Cassandra) est peu adaptée aux mises à jour partielles fréquentes et aux requêtes ad hoc sur des sous-documents ; une base clé-valeur pure (Redis) ne permettrait pas de requêtes riches sur le contenu médical.

5.2 Cassandra — surveillance des objets médicaux connectés

Adéquation au besoin métier — Les équipements médicaux connectés génèrent un flux continu et massif de mesures horodatées (potentiellement des milliers d'écritures par seconde à l'échelle nationale), pour lesquelles la priorité est la disponibilité en écriture et la scalabilité, davantage que la richesse des requêtes.

Principales qualités dans ce contexte :

- Architecture distribuée peer-to-peer sans nœud maître : aucun point unique de défaillance, disponibilité continue même en cas de perte de plusieurs nœuds.
- Écriture optimisée (log structured storage / LSM-tree) offrant un débit d'écriture très supérieur à une base documentaire ou relationnelle sur ce type de charge.
- Scalabilité linéaire par simple ajout de nœuds, sans repartitionnement manuel complexe.
- Modèle en colonnes larges naturellement adapté aux séries temporelles (clé de partition = identifiant du dispositif, clé de clustering = horodatage).

Alternatives écartées et raisons — MongoDB supporterait le volume mais avec un débit d'écriture et une résilience en écriture inférieurs à Cassandra sur ce profil de charge ; une base relationnelle serait rapidement saturée par le volume d'insertions ; une base graphe (Neo4j) n'a pas vocation à absorber de l'écriture massive de mesures brutes.

5.3 Redis — authentification et sessions

Adéquation au besoin métier — L'authentification est sollicitée à chaque requête utilisateur : elle nécessite une latence minimale et une disponibilité quasi absolue, pour un volume de données par utilisateur très faible (jeton, métadonnées de session).

Principales qualités dans ce contexte :

- Stockage en mémoire : temps de réponse de l'ordre du sous-milliseconde.
- Expiration native des clés (TTL), idéale pour la durée de vie des sessions et jetons sans job de nettoyage additionnel.
- Mode cluster et répliquation (Sentinel / Redis Cluster) pour la haute disponibilité.
- Peut également servir de cache applicatif pour soulager MongoDB sur les lectures fréquentes (ex. profils de professionnels de santé).

Alternatives écartées et raisons — Stocker les sessions directement dans MongoDB fonctionnerait mais avec une latence supérieure et une charge d'E/S ajoutée sur la base documentaire, critique pour les usages médicaux ; une base relationnelle serait surdimensionnée pour ce besoin simple et à très fort débit.

5.4 Neo4j — analyse des parcours de soins

Adéquation au besoin métier — L'analyse des parcours de soins consiste essentiellement à parcourir des relations : quels professionnels et établissements un patient a-t-il fréquentés, quels enchaînements de pathologies et de traitements sont observés, etc. Il s'agit d'un problème de traversée de graphe.

Principales qualités dans ce contexte :

- Modèle natif en nœuds et relations : les entités (patient, professionnel, établissement, pathologie) et leurs relations sont représentées explicitement.

- Langage de requête Cypher optimisé pour les traversées multi-sauts (ex. 'patients ayant consulté deux spécialités différentes pour la même pathologie en moins de 3 mois'), avec des temps de réponse indépendants de la taille totale du graphe.
- Facilite les analyses épidémiologiques transversales demandées par le Ministère (ex. détection de clusters, comorbidités fréquentes).

Alternatives écartées et raisons — En documentaire (MongoDB) ou relationnel, ces traversées nécessiteraient des jointures ou des \$lookup en cascade, avec une dégradation de performance à mesure que la profondeur de la requête augmente ; ce n'est pas le cas dans un moteur de graphe natif.

5.5 Elasticsearch — tableaux de bord et indicateurs

Adéquation au besoin métier — Les autorités sanitaires ont besoin d'indicateurs agrégés (nombre de cas par région, tendances épidémiologiques) et de capacités de recherche transversale sur de gros volumes, avec des temps de réponse compatibles avec un usage interactif (dashboards).

Principales qualités dans ce contexte :

- Index inversé optimisé pour la recherche full-text et les filtres combinés à grande échelle.
- Framework d'agrégation performant, conçu pour le calcul d'indicateurs en quasi temps réel.
- Intégration native avec des outils de visualisation (Kibana) pour la production de tableaux de bord sans développement front dédié.
- Scalabilité horizontale par sharding des index, cohérente avec la croissance du volume de données source.

Alternatives écartées et raisons — Faire porter ces agrégations directement sur MongoDB ou Cassandra en production risquerait de dégrader les performances transactionnelles ; une base relationnelle avec un entrepôt de données classique serait plus rigide à faire évoluer et moins performante sur la recherche full-text.

5.6 Apache Kafka — bus événementiel

Adéquation au besoin métier — Le découplage entre l'ingestion massive de données IoT, les services applicatifs et la synchronisation vers Elasticsearch nécessite un composant capable d'absorber les pics de charge et de garantir qu'aucune donnée n'est perdue.

Principales qualités dans ce contexte :

- Découplage producteur/consommateur : chaque service peut évoluer et être redéployé indépendamment.
- Durabilité et rejouabilité des messages : en cas d'indisponibilité momentanée d'un consommateur (ex. maintenance de Cassandra), les données sont conservées et retraitées à la reprise.
- Débit très élevé, adapté aux flux IoT à grande échelle.

Alternatives écartées et raisons — Sans bus événementiel, les services applicatifs devraient interroger directement les objets connectés ou être synchrones avec Cassandra, ce qui créerait un couplage fort et un risque de perte de données en cas de pic de charge ou de panne.

6. Livrable 4 — Disponibilité et sécurité

6.1 Haute disponibilité

- MongoDB : déploiement en replica sets (a minima 3 nœuds répartis sur des zones de disponibilité distinctes) avec basculement (failover) automatique en moins de 30 secondes en cas de perte du nœud primaire.
- Cassandra : facteur de réplication ≥ 3 avec réplication multi-datacenter ; niveau de cohérence configurable (ex. QUORUM) pour arbitrer entre disponibilité et cohérence selon le type d'opération.

- Redis : mode Cluster avec réplication maître-esclave et bascule automatique via Sentinel.
- Neo4j : déploiement en cluster causal (Causal Cluster) avec plusieurs instances de lecture et un mécanisme d'élection de leader.
- Elasticsearch : cluster multi-nœuds avec répartition des shards primaires et de leurs répliques sur des nœuds distincts.
- API Gateway et services applicatifs déployés en plusieurs instances derrière un répartiteur de charge, avec orchestration (ex. Kubernetes) permettant un redémarrage automatique en cas de défaillance.

6.2 Tolérance aux pannes et montée en charge

- Toutes les bases retenues sont nativement distribuées : la perte d'un nœud ne provoque pas d'interruption de service, seulement une dégradation transitoire absorbée par les répliques.
- Montée en charge horizontale : sharding MongoDB (par exemple sur l'identifiant patient ou la région), partitionnement Cassandra (par identifiant de dispositif), ajout de nœuds Elasticsearch — permettant d'accompagner la croissance sans interruption ni refonte.
- Le bus Kafka absorbe les pics de charge en écriture IoT et protège les bases situées en aval d'une saturation brutale.

6.3 Sauvegarde et restauration

- MongoDB : sauvegardes continues (snapshots incrémentaux) et possibilité de restauration point-in-time grâce à l'oplog ; tests de restauration périodiques.
- Cassandra : snapshots réguliers par nœud et export vers un stockage froid pour l'archivage réglementaire.
- Politique de rétention différenciée : conservation longue durée pour les dossiers médicaux (obligations légales de conservation), rétention plus courte et agrégée pour les mesures IoT brutes après traitement.
- Procédure de restauration testée et documentée, avec objectifs de temps de restauration (RTO) et de perte de données maximale tolérée (RPO) définis pour chaque base selon sa criticité.

6.4 Protection des données de santé

- Chiffrement systématique des échanges (TLS) entre tous les composants et chiffrement au repos sur chaque base de données (encryption at rest).
- Chiffrement applicatif (au niveau champ) pour les attributs les plus sensibles du dossier patient (ex. identifiants nationaux, diagnostics), via le chiffrement côté client ou field-level encryption de MongoDB.
- Cloisonnement réseau (VPC, sous-réseaux privés) : les bases de données ne sont jamais exposées directement sur Internet, seule l'API Gateway l'est.
- Anonymisation / pseudonymisation des données transmises aux modules d'analyse épidémiologique et aux tableaux de bord destinés aux autorités sanitaires, conformément au principe de minimisation des données.

6.5 Gestion des utilisateurs et des droits d'accès

- Authentification forte (OAuth2/OIDC, avec authentification multi-facteurs pour les professionnels de santé) déléguée à un fournisseur d'identité dédié.
- Contrôle d'accès basé sur les rôles (RBAC) au niveau de chaque base (rôles MongoDB, Cassandra, etc.) et au niveau applicatif (un médecin n'accède qu'aux dossiers des patients qu'il suit, sauf contexte d'urgence tracé).
- Traçabilité : journalisation systématique des accès et modifications (qui, quoi, quand) dans une collection d'audit dédiée et/ou un flux Kafka immuable, exploitable pour les contrôles réglementaires.

- Révocation immédiate des accès possible via la suppression des sessions Redis associées à un utilisateur.

7. Livrable 5 — Préparation de l'implémentation MongoDB

Le Ministère souhaite débiter le développement par la partie documentaire de l'architecture. Cette section identifie les informations qui seront stockées dans MongoDB, justifie leur nature documentaire, et prépare la modélisation détaillée dans la partie 3 du présent rapport.

7.1 Informations stockées dans MongoDB

- Dossiers patients : identité, coordonnées, antécédents médicaux, allergies, contacts d'urgence.
- Répertoire des établissements de santé partenaires.
- Répertoire des professionnels de santé intervenant dans les actes (identité, spécialité, établissement de rattachement).
- Consultations et hospitalisations, avec leurs prescriptions associées.
- Résultats d'examens médicaux (analyses, imagerie — métadonnées et résultats structurés).

7.2 Pourquoi ces données relèvent d'une base documentaire

- Structure naturellement hiérarchique : un patient possède plusieurs consultations, chacune pouvant comporter plusieurs prescriptions et examens — une représentation en document imbriqué reflète directement ce modèle métier.
- Hétérogénéité des contenus : le contenu d'une consultation ou d'un examen varie fortement selon la spécialité médicale (un compte rendu de cardiologie diffère structurellement d'un compte rendu de radiologie) ; un schéma flexible évite de multiplier les tables et les colonnes optionnelles d'un modèle relationnel.
- Accès applicatif dominant en lecture/écriture par patient ou par consultation : le modèle documentaire permet de restituer un dossier complet, ou une consultation complète, en une seule requête, sans jointure.
- Évolutivité du schéma sans interruption : l'ajout d'un nouveau type d'examen ou d'un nouveau champ métier ne nécessite pas de migration de schéma bloquante, conformément à la contrainte d'évolution progressive des structures de données exprimée dans le cahier des charges.

7.3 Principes retenus pour la modélisation à venir

- Une collection par entité métier disposant d'un cycle de vie propre et d'un volume de croissance distinct (patients, établissements, professionnels, consultations, examens), plutôt qu'une collection unique fourre-tout.
- Imbrication (embedding) des sous-structures bornées et toujours consultées avec leur parent (ex. prescriptions au sein d'une consultation), et référencement (reference) des relations non bornées ou à cycle de vie indépendant (ex. historique des consultations d'un patient), afin d'éviter les documents à croissance illimitée.
- Indexation ciblée sur les champs de recherche fréquents (identifiant patient, identifiant médecin, dates, service) pour garantir des temps de réponse compatibles avec un usage médical quotidien.

Le détail de cette modélisation — collections, structure des documents, choix embedding/reference, index, requêtes et agrégations — est développé dans la partie 3 du présent rapport (Prototype MongoDB).

8. Prototype MongoDB — Modélisation des données

Cette partie détaille la modélisation documentaire qui sera implémentée dans MongoDB, conformément au périmètre défini au livrable 5 : dossiers patients, répertoires (établissements, professionnels), consultations/hospitalisations et examens médicaux.

8.1 Collections retenues

Collection	Description	Justification
patients	Dossier administratif et médical de synthèse de chaque patient (identité, antécédents, allergies, contact d'urgence).	Entité pivot du système, consultée à très haute fréquence et de manière indépendante de son historique de soins ; volume stable par patient (pas de croissance illimitée).
etablissements	Référentiel des établissements de santé partenaires (nom, type, adresse, services proposés).	Référentiel à faible volume et faible fréquence de mise à jour, référencé par les consultations et les professionnels ; séparé pour éviter la duplication.
professionnels	Référentiel des professionnels de santé (identité, numéro RPPS, spécialité, établissement de rattachement).	Cycle de vie propre (arrivée/départ d'un professionnel) indépendant des actes réalisés ; permet de qualifier et d'authentifier les intervenants.
consultations	Consultations et hospitalisations, avec leurs prescriptions imbriquées et les références aux examens demandés.	Cœur du dossier médical ; forte volumétrie (plusieurs centaines de millions à terme) ; accédée principalement par patient ou par médecin.
examens	Résultats d'examens médicaux (biologie, imagerie) rattachés à une consultation.	Cycle de vie et volumétrie propres (un examen peut être produit par un laboratoire externe, consulté indépendamment de la consultation d'origine) ; séparée pour éviter que les documents de consultation ne croissent sans limite.

Relations entre collections : un patient possède plusieurs consultations (1-N) ; une consultation est rattachée à un établissement et à un professionnel (N-1 chacun) et peut donner lieu à plusieurs examens (1-N) ; un professionnel est rattaché à un établissement (N-1).

9. Conception des documents

Pour chaque collection, la structure générale ci-dessous précise les attributs principaux, leur type, et leur caractère obligatoire ou optionnel.

9.1 Collection patients

```
{
  _id: ObjectId,           // obligatoire, généré
  numero_dossier: String,  // obligatoire, unique (ex. "P000001")
  nom: String,             // obligatoire
  prenom: String,          // obligatoire
  date_naissance: Date,    // obligatoire
  sexe: String,            // obligatoire ("M" | "F" | "Autre")
  adresse: {              // optionnel
    rue: String,
    ville: String,
    code_postal: String
  },
  telephone: String,       // optionnel
  email: String,           // optionnel
  groupe_sanguin: String,  // optionnel
  allergies: [String],     // optionnel, liste bornée
  antecedents: [          // optionnel, liste bornée
```

```

    { libelle: String, date: Date }
  ],
  contact_urgence: {                // optionnel
    nom: String, telephone: String
  },
  date_creation: Date                // obligatoire
}

```

9.2 Collection etablissements

```

{
  _id: ObjectId,                    // obligatoire
  nom: String,                      // obligatoire
  type: String,                     // obligatoire
  ("hopital"|"clinique"|"cabinet"|"laboratoire")
  adresse: { rue, ville, code_postal }, // optionnel
  services: [String],               // optionnel (ex. ["Cardiologie","Urgences"])
  telephone: String,                // optionnel
  date_adhesion: Date               // obligatoire
}

```

9.3 Collection professionnels

```

{
  _id: ObjectId,                    // obligatoire
  numero_rpps: String,              // obligatoire, unique
  nom: String,                      // obligatoire
  prenom: String,                   // obligatoire
  specialite: String,               // obligatoire
  etablissement_id: ObjectId,        // obligatoire (référence -> etablissements)
  email: String,                    // optionnel
  telephone: String,                // optionnel
  actif: Boolean                     // obligatoire (défaut: true)
}

```

9.4 Collection consultations

```

{
  _id: ObjectId,                    // obligatoire
  patient_id: ObjectId,              // obligatoire (référence -> patients)
  medecin: {                         // obligatoire (référence enrichie -> professionnels)
    id: ObjectId,
    nom: String,
    prenom: String,
    specialite: String
  },
  etablissement_id: ObjectId,         // obligatoire (référence -> etablissements)
  type: String,                       // obligatoire ("consultation"|"hospitalisation")
  service: String,                    // obligatoire (ex. "Cardiologie")
  date_debut: Date,                   // obligatoire
  date_fin: Date,                     // optionnel (hospitalisation en cours = absent)
  motif: String,                      // obligatoire
  diagnostic: String,                 // optionnel
  prescriptions: [                    // optionnel, liste bornée (embedding)
    { medicament: String, dosage: String, duree_jours: Number }
  ],
  examens_ids: [ObjectId],            // optionnel, liste de références -> examens
  statut: String                      // obligatoire ("en_cours"|"terminee")
}

```

9.5 Collection examens

```

{
  _id: ObjectId,                    // obligatoire
  patient_id: ObjectId,              // obligatoire (référence -> patients)
  consultation_id: ObjectId,          // obligatoire (référence -> consultations)
}

```

```

type_examen: String,           // obligatoire ("biologie"|"imagerie"|"autre")
date_examen: Date,            // obligatoire
etablissement_id: ObjectId,    // obligatoire (laboratoire / plateau technique)
resultats: [                  // optionnel, liste bornée (embedding)
  { parametre: String, valeur: String, unite: String, valeur_reference: String }
],
compte_rendu: String,         // optionnel
fichier_url: String           // optionnel (lien vers imagerie stockée hors MongoDB)
}

```

10. Justification de la modélisation (Embedding / Reference)

Relation	Embed / Reference	Justification
Patient → allergies / antécédents	Embedding	Liste courte et bornée, systématiquement lue avec le patient, sans cycle de vie propre : l'embedding évite une requête supplémentaire.
Patient → consultations	Reference	Relation 1-N non bornée (des centaines de consultations sur une vie de patient) : l'embedding ferait croître le document patient au-delà du raisonnable et risquerait d'atteindre la limite de 16 Mo ; la référence permet aussi de consulter une consultation indépendamment du patient.
Consultation → prescriptions	Embedding	Liste courte et bornée (quelques lignes par consultation), toujours consultée avec la consultation, jamais interrogée seule : l'embedding garantit l'atomicité de la mise à jour et évite une jointure.
Consultation → médecin (professionnel)	Reference enrichie (référence + copie de champs d'affichage)	Le professionnel a un cycle de vie propre (changement d'établissement, désactivation) : on référence son _id pour garantir l'intégrité, tout en dupliquant nom/prénom/spécialité dans la consultation pour éviter une jointure systématique à l'affichage de l'historique (compromis lecture/cohérence assumé et documenté).
Consultation → établissement	Reference	Référentiel à faible volume mais partagé par un très grand nombre de consultations : la référence évite la duplication et permet de mettre à jour un établissement en un seul endroit.
Consultation → examens	Reference	Les examens ont un cycle de vie et une volumétrie propres (résultats ajoutés après coup par un laboratoire, parfois avec des fichiers volumineux) : les référencer évite de faire grossir indéfiniment le document de consultation et permet de les interroger indépendamment (ex. par laboratoire).
Examen → résultats (paramètres mesurés)	Embedding	Liste bornée de paramètres toujours consultée avec l'examen auquel elle appartient : aucune raison de la séparer.
Professionnel → établissement	Reference	Référentiel partagé, mis à jour indépendamment des professionnels qui y sont rattachés.

11. Optimisation — Index

Les index proposés ci-dessous ciblent les champs effectivement utilisés par les requêtes attendues (section 12) et par les vues métier à fort trafic. Chaque index supplémentaire ayant un coût en écriture (mise à jour de la structure d'index à chaque insertion/modification) et en espace de stockage, seuls les champs à fort bénéfice de lecture ont été retenus — une indexation systématique de tous les champs serait contre-productive sur une collection à très fort volume d'écriture comme consultations.

Index proposé	Besoin métier	Gain attendu	Conséquences sur les écritures
patients : { numero_dossier: 1 } (unique)	Rechercher un patient à partir de son identifiant (requête n°1).	Recherche en $O(\log n)$ au lieu d'un balayage complet de la collection ; contrainte d'unicité garantie par la base.	Coût marginal à l'insertion (mise à jour d'un index B-tree), négligeable au regard du gain en lecture, fréquence d'insertion patient faible par rapport aux lectures.
patients : { nom: 1, prenom: 1, date_naissance: 1 }	Rechercher un patient par identité (accueil, secrétariat) sans connaître son numéro de dossier.	Évite un balayage complet sur un critère de recherche fréquent côté accueil des établissements.	Index composé supplémentaire à maintenir ; impact faible car les mises à jour de ces champs sont rares.
consultations : { patient_id: 1, date_debut: -1 }	Afficher les consultations d'un patient, triées par date décroissante (requête n°2).	Accès direct aux consultations d'un patient sans balayage, tri déjà pris en charge par l'ordre de l'index.	Index à maintenir sur une collection à très forte volumétrie et fort taux d'écriture : coût non négligeable mais indispensable, la requête étant parmi les plus fréquentes du système.
consultations : { "medecin.id": 1, date_debut: -1 }	Afficher les consultations réalisées par un médecin (requête n°3), utile aussi pour les statistiques d'activité.	Recherche directe par praticien sans balayage de l'ensemble des consultations.	Coût d'écriture supplémentaire similaire au précédent ; acceptable car ce champ n'est jamais modifié après création.
consultations : { service: 1, statut: 1 }	Rechercher les patients hospitalisés dans un service donné (requête n°4).	Filtre direct sur service + statut, essentiel pour la vue "lits occupés" consultée en continu par les services hospitaliers.	Champ statut mis à jour à la sortie du patient : chaque changement de statut implique une mise à jour de l'index, mais le volume de changements est très inférieur au volume de lectures de cette vue.
examens : { date_examen: 1 }	Rechercher les examens réalisés durant une période donnée (requête n°5).	Filtre par plage de dates efficace pour les extractions et les analyses épidémiologiques.	Coût d'écriture modéré, la date d'examen étant fixée une seule fois à la création du document.
examens : { patient_id: 1 }	Retrouver rapidement tous les examens d'un patient (complément de la vue dossier patient).	Évite le balayage complet de la collection examens, dont le volume croît plus vite que celui des patients.	Impact d'écriture faible, champ non modifié après création.

12. Requêtes MongoDB

Les requêtes ci-dessous répondent aux besoins fonctionnels listés dans le cahier des charges. Elles sont également fournies, prêtes à l'exécution, dans le script `requetes.js` livré avec le prototype.

12.1 Rechercher un patient à partir de son identifiant

```
db.patients.findOne({ numero_dossier: "P000001" });
```

12.2 Afficher les consultations d'un patient

```
db.consultations.find(
  { patient_id: ObjectId("PATIENT_ID") }
).sort({ date_debut: -1 });
```

12.3 Afficher les consultations réalisées par un médecin

```
db.consultations.find(
  { "medecin.id": ObjectId("MEDECIN_ID") }
).sort({ date_debut: -1 });
```

12.4 Rechercher les patients hospitalisés dans un service donné

La collection `consultations` porte les informations de service et de statut ; une agrégation avec `$lookup` permet de restituer directement l'identité des patients concernés :

```
db.consultations.aggregate([
  { $match: { type: "hospitalisation", service: "Cardiologie", statut: "en_cours" } },
  { $lookup: {
    from: "patients",
    localField: "patient_id",
    foreignField: "_id",
    as: "patient"
  } },
  { $unwind: "$patient" },
  { $project: {
    _id: 0,
    numero_dossier: "$patient.numero_dossier",
    nom: "$patient.nom",
    prenom: "$patient.prenom",
    date_debut: 1
  } }
]);
```

12.5 Rechercher les examens réalisés durant une période donnée

```
db.examens.find({
  date_examen: {
    $gte: ISODate("2026-01-01T00:00:00Z"),
    $lte: ISODate("2026-03-31T23:59:59Z")
  }
});
```

13. Agrégations

Trois analyses ont été réalisées à l'aide du framework d'agrégation, au-delà du minimum de deux demandé, afin d'illustrer différents besoins de pilotage de la plateforme.

13.1 Nombre de consultations par établissement

```
db.consultations.aggregate([
  { $group: { _id: "$etablissement_id", nombre_consultations: { $sum: 1 } } },
  { $sort: { nombre_consultations: -1 } },
  { $lookup: {
    from: "etablissements",
```

```

    localField: "_id",
    foreignField: "_id",
    as: "etablissement"
  }},
  { $unwind: "$etablissement" },
  { $project: {
    _id: 0,
    etablissement: "$etablissement.nom",
    nombre_consultations: 1
  }}
]);

```

13.2 Répartition des examens par type

```

db.examens.aggregate([
  { $group: { _id: "$type_examen", total: { $sum: 1 } } },
  { $sort: { total: -1 } }
]);

```

13.3 Nombre moyen de consultations par médecin

```

db.consultations.aggregate([
  { $group: { _id: "$medecin.id", nb_consultations: { $sum: 1 } } },
  { $group: { _id: null, moyenne_consultations_par_medecin: { $avg: "$nb_consultations" } } }
]);

```

14. Validation finale — Conclusion

En quoi le modèle MongoDB répond-il aux besoins exprimés dans le cahier des charges ?

Le modèle proposé respecte la structure hiérarchique naturelle du dossier médical (patient → consultations → prescriptions/examens) tout en évitant les documents à croissance illimitée grâce à un usage raisonné du référencement. Il permet un accès rapide aux informations d'un patient ou d'un médecin (requêtes indexées, sans jointure pour les données fréquemment consultées ensemble), supporte l'ajout de nouveaux établissements et de nouveaux types d'examens sans modification de schéma, et s'appuie sur les mécanismes natifs de MongoDB (replica sets, sharding) pour répondre aux exigences de disponibilité continue et de montée en charge du cahier des charges.

Quels sont les avantages de cette modélisation ?

- Lecture efficace des cas d'usage les plus fréquents (dossier patient, historique de consultations) grâce à l'embedding ciblé des sous-structures bornées.
- Flexibilité du schéma, essentielle face à l'hétérogénéité des spécialités médicales et à l'évolution attendue de la plateforme sur plusieurs années.
- Séparation claire des entités à cycle de vie et à volumétrie différents (patients, professionnels, établissements, consultations, examens), ce qui facilite le sharding futur (par exemple sur patient_id) sans document surdimensionné.
- Index ciblés sur les requêtes réellement exprimées dans le cahier des charges, limitant le surcoût en écriture tout en garantissant des temps de réponse compatibles avec un usage médical quotidien.

Quelles sont ses limites ?

- La duplication partielle des données du médecin dans chaque consultation (référence enrichie) introduit un risque de désynchronisation si un professionnel change de nom ou de spécialité ; une procédure de mise à jour en masse (ou l'acceptation d'un léger historique figé, ce qui est d'ailleurs cohérent avec la traçabilité médico-légale) doit être formalisée.
- L'absence de jointures natives puissantes (comparées à un moteur relationnel) rend certaines analyses transversales complexes plus coûteuses à exprimer, ce qui justifie le renvoi de ces analyses vers Neo4j et Elasticsearch dans l'architecture globale plutôt que de tout faire porter par MongoDB.

- Le référencement systématique des examens et consultations impose une discipline applicative (gestion explicite de la cohérence entre collections, MongoDB ne garantissant pas de contrainte d'intégrité référentielle native comme une clé étrangère relationnelle).

Quelles évolutions envisager si le projet se développe dans les prochaines années ?

- Mise en place effective du sharding (par exemple sur patient_id ou sur une clé géographique/régionale) dès que la volumétrie ou la charge d'écriture l'exige, pour anticiper la croissance vers les 25 millions de dossiers annoncés.
- Externalisation des pièces volumineuses (imagerie médicale) vers un stockage objet dédié (le champ fichier_url du modèle actuel anticipe déjà ce besoin), MongoDB restant réservé aux métadonnées structurées.
- Renforcement du chiffrement au niveau champ pour les données les plus sensibles, à mesure que le référentiel réglementaire se précise.
- Mise en place de vues matérialisées ou d'une synchronisation plus fine vers Elasticsearch pour soulager davantage MongoDB des besoins purement analytiques, si le volume de tableaux de bord venait à augmenter significativement.